

엔코더 모터에 대한 조사

1. 수행목표

엔코더의 개념과 사용법을 학습하고, 로봇 운반차에서 엔코더 모터를 이용하여 바퀴의 회전 수, 속도, 이동 거리 등을 계산하는 방법을 이해한다. 또한 엔코더 데이터와 실제 주행 결과 사이에 차이가 발생하는 이유와 이를 보정하는 방법을 조사한다.

2. 엔코더 모터의 정의

엔코더 모터란 일반 모터에 엔코더가 결합된 형태의 모터를 말한다. 모터는 전기 에너지를 회전 운동으로 바꾸는 장치이고, 엔코더는 모터 축이나 바퀴가 얼마나 회전했는지 측정하는 센서이다.

즉, 엔코더 모터는 단순히 회전만 하는 모터가 아니라, 회전한 정도를 전기적 신호로 출력할 수 있는 모터이다. 로봇에서는 이 신호를 이용해 바퀴가 몇 바퀴 돌았는지, 로봇이 얼마나 이동했는지, 현재 속도가 어느 정도인지 계산할 수 있다.

엔코더 모터는 다음과 같은 분야에서 많이 사용된다.

사용 분야	활용 목적
로봇 운반차	이동 거리, 속도, 방향 제어
모바일 로봇	위치 추정, 주행 제어
컨베이어 시스템	이동량 측정, 속도 제어
CNC 장비	축 위치 제어
서보 시스템	위치 피드백 제어

로봇 운반차에서는 엔코더 모터를 사용하면 단순히 모터에 전압을 주어 움직이는 방식보다 더 정확한 주행 제어가 가능하다.

3. 엔코더 부의 역할

3.1 엔코더 부의 역할

엔코더 부는 모터의 회전 상태를 측정하여 제어기에게 전달하는 역할을 한다. 여기서 제어기는 보통 아두이노, 라즈베리파이, Jetson, STM32, Teensy 같은 마이크로컨트롤러 또는 임베디드 보드가 될 수 있다.

엔코더 부가 하는 역할은 다음과 같다.

역할	설명
회전량 측정	모터 축이나 바퀴가 얼마나 회전했는지 측정한다.
회전 방향 판단	A상, B상 신호의 순서를 이용해 정방향/역방향을 판단한다.
속도 계산	일정 시간 동안 들어온 펄스 수를 이용해 회전 속도를 계산한다.
위치 추정	바퀴 회전량을 누적하여 로봇의 이동 거리와 위치를 추정한다.
피드백 제어	목표 속도와 실제 속도의 차이를 줄이는 제어에 사용된다.

엔코더는 로봇이 실제로 얼마나 움직였는지 확인하는 피드백 센서이기 때문에, 정확한 주행 제어에서 매우 중요하다.

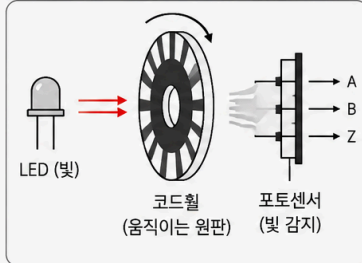
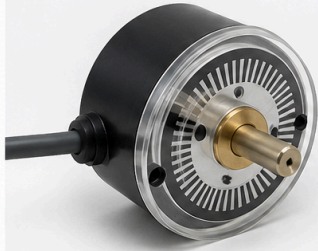
3.2 엔코더 부의 구성 방식

엔코더는 측정 방식에 따라 여러 종류가 있지만, 로봇 모터에서는 주로 광학식 엔코더와 자기식 엔코더가 많이 사용된다.

구분	구성	작동 방식	특징
광학식 엔코더	회전 디스크, LED, 포토 센서	디스크의 구멍이나 패턴을 빛으로 감지	정밀도가 높지만 먼지와 오염에 약할 수 있다.
자기식 엔코더	자석, 홀 센서	자석의 N극/S극 변화를 감지	구조가 비교적 단순하고 먼지에 강하다.
접촉식 엔코더	브러시, 접점 패턴	접점이 닿는 위치에 따라 신호 발생	구조는 단순하지만 마모가 발생할 수 있다.

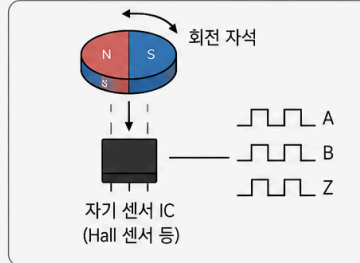
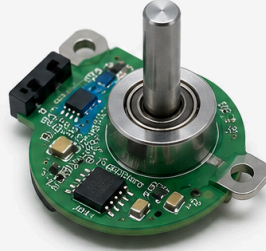
소형 로봇이나 DC 기어드 모터에서는 자기식 엔코더가 많이 사용된다. 모터 축에 작은 자석이 붙어 있고, 자석이 회전하면 홀 센서가 자기장의 변화를 감지하여 펄스 신호를 만든다.

광학식 엔코더 (Optical Encoder)



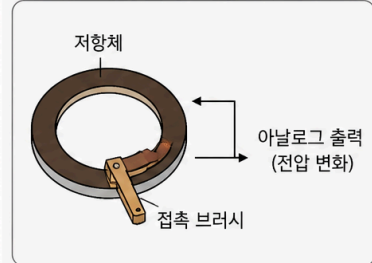
- 빛(LED)을 이용해 회전하는 원판의 슬롯을 통과하는 빛을 감지
- 고해상도, 정밀도 우수
- 먼지, 오염에 민감
- 주로 산업용, 로봇, 정밀장비에 사용

자기식 엔코더 (Magnetic Encoder)



- 자석의 자기장을 센서로 감지
- 먼지, 오염, 진동에 강함
- 광학식보다 해상도는 낮지만 내구성 우수
- 자동차, 산업장비, 가전제품 등에 사용

접촉식 엔코더 (Contact Encoder)



- 브러시가 저항체에 직접 접촉하여 위치 검출
- 구조가 단순하고 저렴
- 마모, 수명 짧음, 정밀도 낮음
- 일반 가전, 오디오 볼륨 조절 등에 사용

3.3 엔코더의 작동 원리

엔코더는 회전 운동을 전기적 펄스 신호로 바꾸는 장치이다. 모터 축이 회전하면 엔코더 내부의 디스크나 자석도 함께 회전한다. 이때 센서가 회전 패턴을 감지하고, 일정 각도마다 HIGH/LOW 신호를 출력한다. 이 HIGH/LOW 신호가 반복되면 펄스가 만들어진다. 제어기는 이 펄스의 개수를 세어 회전량을 계산한다. 예를 들어, 엔코더가 한 바퀴에 20개의 펄스를 출력한다면 다음과 같이 해석할 수 있다.

펄스 수	회전량
20펄스	모터 축 1바퀴 회전
10펄스	모터 축 0.5바퀴 회전
40펄스	모터 축 2바퀴 회전

엔코더가 A상과 B상 두 개의 신호를 출력하는 경우도 많다. 이를 쿼드러처 엔코더라고 한다. A상과 B상은 서로 약간의 위상 차이를 가지고 출력된다. 제어기는 두 신호 중 어느 신호가 먼저 변하는지를 확인하여 회전 방향을 판단할 수 있다.

ex-1)

A상이 B상보다 먼저 변하면 정방향, B상이 A상보다 먼저 변하면 역방향으로 판단하는 방식이다.

ex-2) -실무기준 로봇이 앞으로 갈 때 엔코더 카운트가 증가하면 정방향 로봇이 뒤로 갈 때 엔코더 카운트가 감소하면 역방향

4. 펄스와 해상도의 개념 및 관계

4.1 펄스의 개념

펄스는 엔코더가 출력하는 디지털 신호의 한 번의 변화 또는 반복 신호를 의미한다. 엔코더는 회전할 때마다 일정한 간격으로 펄스를 출력한다. 제어기는 이 펄스를 카운트하여 회전량을 계산한다.

펄스 수가 많을수록 더 세밀하게 회전량을 측정할 수 있다.

4.2 해상도의 개념

해상도는 엔코더가 회전을 얼마나 세밀하게 나누어 측정할 수 있는지를 의미한다. 보통 PPR 또는 CPR이라는 단위로 표현한다.

중요

펄스 = HIGH로 튀어오른 신호 부분 사이클 = LOW → HIGH → LOW 전체 반복

용어	의미
PPR	Pulse Per Revolution, 1회전당 발생하는 펄스 수
CPR	Count Per Revolution 또는 Cycle Per Revolution으로 사용되며, 문맥에 따라 의미를 확인해야 한다. 한 바퀴 회전할 때 발생하는 사이클 수
해상도	회전 한 바퀴를 얼마나 작은 단위로 나누어 측정할 수 있는지 나타내는 값

용어	쉬운 의미	예시
PPR (Pulses Per Revolution)	엔코더 축이 한 바퀴 돌 때 발생하는 펄스 수	A상 HIGH 펄스가 100개 발생하면 100 PPR
CPR (Cycles Per Revolution)	엔코더 축이 한 바퀴 돌 때 신호 주기가 몇 번 반복되는지	A상 파형이 100번 반복되면 100 CPR
Counts Per Revolution	제어기나 프로그램이 실제로 세는 카운트 수	1체배는 100카운트, 2체배는 200카운트, 4체배는 400카운트

PPR = 깃발이 몇 개 지나갔는지 CPR = 깃발 신호 패턴이 몇 번 반복됐는지 Counts Per Revolution = 제어기가 실제로 몇 번 션는지

PPR은 깃발 하나하나가 지나가는 펄스 수이다. CPR은 바퀴가 한 바퀴 돌 때 신호 패턴이 몇 번 반복되는지이다. Counts Per Revolution은 제어기가 실제로 카운팅한 수이다.

예를 들어 1회전에 100개의 펄스를 출력하는 엔코더는 1펄스당 3.6도씩 회전한 것으로 볼 수 있다.

1펄스당 각도 = $360\text{도} / \text{PPR}$
1펄스당 각도 = $360\text{도} / 100 = 3.6\text{도}$

만약 PPR이 1000이라면 1펄스당 각도는 0.36도가 된다. 따라서 PPR이 높을수록 더 정밀한 측정이 가능하다.

4.3 펄스와 해상도의 관계

펄스와 해상도는 직접적인 관계가 있다. 한 바퀴를 돌 때 발생하는 펄스 수가 많을수록 해상도가 높다.

PPR	1펄스당 회전 각도	특징
20 PPR	18도	해상도가 낮아 정밀 제어가 어렵다.
100 PPR	3.6도	일반적인 속도 측정에 사용할 수 있다.
500 PPR	0.72도	비교적 정밀한 위치 제어가 가능하다.
1000 PPR	0.36도	정밀 제어에 유리하다.

또한 A상과 B상을 모두 사용하는 쿼드러처 엔코더에서는 신호를 읽는 방식에 따라 카운트 수가 증가할 수 있다.

읽는 방식	설명	카운트 수
1체배	A상의 상승 에지만 카운트	기본 PPR
2체배	A상의 상승/하강 에지를 카운트	2배
4체배	A상과 B상의 상승/하강 에지를 모두 카운트	4배

예를 들어 100 PPR 엔코더를 4체배로 읽으면 실제 제어기에서 1회전당 400카운트로 사용할 수 있다. 이 경우 더 정밀한 회전 측정이 가능하다.

5. 엔코더 데이터를 활용한 속도 계산 방법

5.1 회전 속도 계산

엔코더로 속도를 계산하려면 일정 시간 동안 들어온 펄스 수를 측정하면 된다.

기본 공식은 다음과 같다.

$$\text{초당 회전수(RPS)} = \text{일정 시간 동안 측정한 펄스 수} / (\text{PPR} \times \text{측정 시간})$$
$$\text{분당 회전수(RPM)} = \text{RPS} \times 60$$

예를 들어 PPR이 100인 엔코더에서 1초 동안 50개의 펄스가 측정되었다면 다음과 같다.

$$\text{RPS} = 50 / (100 \times 1) = 0.5\text{회전/초}$$
$$\text{RPM} = 0.5 \times 60 = 30\text{RPM}$$

즉, 모터는 1분에 30회전하고 있는 것이다.

5.2 바퀴 선속도 계산

로봇 운반차에서는 모터의 회전 속도보다 바퀴가 실제로 바닥 위를 얼마나 빠르게 이동하는지가 중요하다. 이를 선속도라고 한다.

바퀴의 둘레는 다음과 같이 계산한다.

$$\text{바퀴 둘레} = \pi \times \text{바퀴 지름}$$

그리고 바퀴 선속도는 다음과 같이 계산한다.

$$\text{선속도} = \text{초당 회전수(RPS)} \times \text{바퀴 둘레}$$

예를 들어 바퀴 지름이 0.1m이고, 바퀴가 초당 2회전한다면 다음과 같다.

$$\begin{aligned}\text{바퀴 둘레} &= 3.14 \times 0.1 = 0.314\text{m} \\ \text{선속도} &= 2 \times 0.314 = 0.628\text{m/s}\end{aligned}$$

따라서 로봇은 이론적으로 초당 약 0.628m 이동한다.

6. 엔코더 데이터를 활용한 거리 계산 방법

6.1 이동 거리 계산

엔코더로 이동 거리를 계산하려면 누적 펄스 수를 이용한다.

기본 공식은 다음과 같다.

$$\begin{aligned}\text{바퀴 회전수} &= \text{누적 펄스 수} / \text{PPR} \\ \text{이동 거리} &= \text{바퀴 회전수} \times \text{바퀴 둘레}\end{aligned}$$

즉, 한 펄스당 이동 거리를 구한 뒤 누적 펄스 수에 곱해도 된다.

$$\begin{aligned}\text{1펄스당 이동 거리} &= \text{바퀴 둘레} / \text{PPR} \\ \text{이동 거리} &= \text{누적 펄스 수} \times \text{1펄스당 이동 거리}\end{aligned}$$

예를 들어 바퀴 지름이 0.1m이고 PPR이 100인 엔코더가 있다고 가정한다.

$$\begin{aligned}\text{바퀴 둘레} &= 3.14 \times 0.1 = 0.314\text{m} \\ \text{1펄스당 이동 거리} &= 0.314 / 100 = 0.00314\text{m}\end{aligned}$$

이 상태에서 500개의 펄스가 측정되었다면 다음과 같다.

$$\text{이동 거리} = 500 \times 0.00314 = 1.57\text{m}$$

따라서 로봇은 이론적으로 약 1.57m 이동한 것으로 계산할 수 있다.

6.2 기어비가 있는 경우

엔코더가 모터 축에 달려 있고 바퀴는 감속기 뒤쪽에 연결되어 있다면 기어비를 고려해야 한다.

예를 들어 모터축 엔코더가 1회전당 20펄스를 출력하고, 감속비가 1:30이라면 바퀴가 1회전하기 위해 모터축은 30회전해야 한다.

바퀴 1회전당 펄스 수 = 모터축 PPR × 감속비
바퀴 1회전당 펄스 수 = $20 \times 30 = 600$ 펄스

따라서 실제 바퀴 이동 거리를 계산할 때는 모터축 PPR이 아니라 바퀴 1회전당 펄스 수를 기준으로 계산해야 한다.

7. 엔코더 데이터와 실제 주행 결과의 차이가 발생하는 이유

엔코더 데이터는 바퀴가 얼마나 회전했는지를 측정한다. 하지만 바퀴가 회전한 양과 로봇이 실제로 이동한 거리가 항상 정확히 일치하는 것은 아니다.

대표적인 오차 원인은 다음과 같다.

오차 원인	설명
바퀴 미끄러짐	바퀴가 회전했지만 바닥에서 미끄러지면 실제 이동 거리는 계산값보다 짧아진다.
바퀴 지름 오차	실제 바퀴 지름이 계산에 사용한 값과 다르면 거리 계산에 오차가 생긴다.
타이어 변형	하중 때문에 바퀴가 눌리면 실제 유효 반지름이 달라진다.
바닥 상태	먼지, 경사, 요철, 미끄러운 바닥에서 오차가 커진다.
좌우 바퀴 차이	좌우 바퀴의 지름이나 마찰 상태가 다르면 직진이 틀어진다.
기어 백래시	기어 사이의 유격 때문에 회전 방향 전환 시 오차가 발생할 수 있다.
엔코더 신호 누락	빠른 회전이나 노이즈로 인해 펄스를 제대로 읽지 못할 수 있다.
샘플링 시간 오차	속도 계산 시 측정 시간이 부정확하면 속도 값이 흔들릴 수 있다.

특히 모바일 로봇에서는 바퀴 미끄러짐이 큰 오차 원인이 된다. 엔코더는 바퀴가 회전했다는 사실은 알 수 있지만, 바퀴가 실제로 바닥을 밀고 로봇을 이동시켰는지는 직접 알 수 없기 때문이다.

8. 엔코더 오차 보정 방법

8.1 바퀴 지름 보정

바퀴 지름을 자로 측정한 값만 사용하면 오차가 생길 수 있다. 실제 주행 실험을 통해 유효 바퀴 지름을 보정할 수 있다.

예를 들어 로봇에게 1m를 이동하라고 명령했는데 실제로 0.95m만 이동했다면 계산에 사용한 바퀴 지름이나 펄스당 이동 거리를 조정해야 한다.

보정 계수 = 실제 이동 거리 / 계산 이동 거리

이 보정 계수를 이동 거리 계산식에 곱하면 오차를 줄일 수 있다.

8.2 좌우 바퀴 보정

차동 구동 로봇에서는 좌우 바퀴의 차이가 크면 직진 명령을 내려도 한쪽으로 휘어진다. 이 경우 좌우 바퀴의 펄스당 이동 거리 또는 속도 명령을 따로 보정해야 한다.

예를 들어 오른쪽으로 휘어진다면 왼쪽 바퀴가 상대적으로 더 많이 이동했거나 오른쪽 바퀴가 덜 이동했을 가능성이 있다. 이때 오른쪽 바퀴의 속도를 약간 높이거나 왼쪽 바퀴의 속도를 약간 낮추는 방식으로 보정할 수 있다.

8.3 반복 주행 실험을 통한 보정

엔코더 보정은 한 번의 실험으로 끝내기보다 여러 번 반복하여 평균값을 사용하는 것이 좋다.

예시 보정 절차는 다음과 같다.

1. 바닥에 1m 또는 2m 기준선을 표시한다.
2. 로봇에게 기준 거리만큼 직진 명령을 내린다.
3. 실제 이동 거리를 측정한다.
4. 계산 이동 거리와 실제 이동 거리의 차이를 기록한다.
5. 여러 번 반복하여 평균 오차를 구한다.
6. 보정 계수를 계산식에 적용한다.

이 과정을 통해 엔코더 기반 거리 계산의 정확도를 높일 수 있다.

8.4 다른 센서와의 융합

엔코더만 사용하면 바퀴 미끄러짐이나 바닥 상태 변화에 약하다. 따라서 더 정확한 위치 추정을 위해 다른 센서와 함께 사용하는 경우가 많다.

센서	보완 역할
IMU	회전 각도, 기울기, 각속도 보정
LiDAR	주변 지도와 비교하여 위치 보정
카메라	시각 정보를 이용한 위치 추정
GPS	실외에서 전역 위치 보정
라인 센서	정해진 경로 주행 보정

로봇에서는 엔코더 데이터를 단독으로 사용하기보다 IMU, LiDAR, 카메라 등의 센서와 함께 사용하면 더 안정적인 주행이 가능하다.

9. 로봇 운반차에서 엔코더 모터를 사용할 때 고려사항

로봇 운반차에 엔코더 모터를 사용할 때는 다음 사항을 고려해야 한다.

고려사항	설명
PPR	필요한 위치 정밀도에 맞는 해상도를 선택해야 한다.
기어비	모터축 기준 PPR과 바퀴축 기준 PPR을 구분해야 한다.
바퀴 지름	거리 계산에 직접 영향을 주므로 정확히 측정해야 한다.
제어 주기	너무 느리면 속도 계산이 부정확해질 수 있다.
인터럽트 사용	빠른 펄스를 놓치지 않기 위해 인터럽트를 사용하는 것이 좋다.
노이즈 대책	배선 정리, 풀업 저항, 필터링 등을 고려해야 한다.
바닥 환경	미끄러운 바닥에서는 엔코더 오차가 커질 수 있다.
하중 변화	적재물 무게에 따라 바퀴 눌림과 미끄러짐이 달라질 수 있다.

인터럽트 사용이유

실생활 비유

인터럽트 없이 엔코더를 읽는 건:

문 앞을 사람이 지나는지 내가 계속 눈으로 확인하는 방식

인터럽트로 읽는 건:

문에 자동 센서를 달아놓고 사람이 지나갈 때마다 알림이 울리는 방식

자동 센서가 있으면 내가 다른 일을 하고 있어도 사람이 지나간 것을 놓치지 않겠지?

엔코더 인터럽트도 똑같아. 모터가 빠르게 돌 때 펄스를 놓치지 않게 해 주는 역할을 해.

인터럽트 없이 읽는 경우 `if (digitalRead(encoderA) == HIGH) { count++; }`

인터럽트 쓰는 경우 `attachInterrupt(digitalPinToInterrupt(2), encoderISR, RISING);`

`void encoderISR() { count++; }`

10. 결론

엔코더 모터는 모터에 엔코더가 결합된 장치로, 모터의 회전량과 회전 방향을 측정할 수 있다. 로봇 운반차에서는 엔코더 데이터를 이용하여 바퀴의 회전 수, 속도, 이동 거리 등을 계산할 수 있으며, 이를 통해 보다 정확한 주행 제어가 가능하다.

엔코더는 펄스 신호를 출력하고, 제어기는 이 펄스를 카운트하여 회전량을 계산한다. PPR이 높을수록 해상도가 높아져 더 세밀한 측정이 가능하다. 또한 A상과 B상을 사용하는 쿼드러처 엔코더는 회전 방향 판단과 고해상도 측정에 유리하다.

하지만 엔코더 데이터와 실제 주행 결과는 항상 일치하지 않는다. 바퀴 미끄러짐, 바퀴 지름 오차, 바닥 상태, 하중 변화, 기어 백래시, 신호 누락 등으로 인해 오차가 발생할 수 있다. 따라서 실제 주행 실험을 통해

보정 계수를 구하고, 필요하면 IMU, LiDAR, 카메라 등의 센서와 함께 사용해야 한다.

결론적으로 엔코더 모터는 로봇 운반차의 속도 제어와 거리 계산에 매우 중요한 부품이며, 정확한 사용을 위해서는 펄스, 해상도, 기어비, 바퀴 지름, 오차 보정 방법을 함께 이해해야 한다.

11. 참고자료

소제목	참고 링크
2. 엔코더 모터의 정의	https://www.dynapar.com/technology/encoder_basics/
3.1 엔코더 부의 역할	https://www.omchsmps.com/what-is-rotary-encoder/
3.2 엔코더 부의 구성 방식	https://www.digikey.com/en/blog/understanding-magnetic-encoders
3.3 엔코더의 작동 원리	https://www.usdigital.com/news/blog/what-is-quadrature/
4.1 펄스의 개념	https://www.dynapar.com/technology/encoder_basics/
4.2 해상도의 개념	https://en.wikipedia.org/wiki/Incremental_encoder
4.3 펄스와 해상도의 관계	https://www.usdigital.com/news/blog/what-is-quadrature/
5. 엔코더 데이터를 활용한 속도 계산 방법	https://www.motioncontroltips.com/how-are-encoders-used-for-speed-measurement/
6. 엔코더 데이터를 활용한 거리 계산 방법	https://zbotic.in/encoder-wheel-odometry-robot-distance-position-arduino/
7. 엔코더 데이터와 실제 주행 결과의 차이가 발생하는 이유	https://blogs.ntu.edu.sg/scemdp-1920s2-g16/
8. 엔코더 오차 보정 방법	https://www-personal.umich.edu/~johannb/Papers/paper58.pdf
9. 로봇 운반차에서 엔코더 모터를 사용할 때 고려사항	https://control.com/technical-articles/an-introduction-to-motor-encoder-types/

6_differential_drive.md

차동 구동 방식에 대한 조사

1. 수행목표

두 개의 바퀴를 가진 이동 로봇이 방향을 바꾸는 방식 중 하나인 차동 구동 방식을 이해한다. 이를 위해 라디안의 개념, 각속도와 회전 반경, 좌우 바퀴 속도 차이에 따른 이동 경로 변화, 제자리 회전 방법, 바퀴 간 거리인 휠 베이스가 주행에 미치는 영향, 회전 반경 계산 방법을 학습한다.

2. 라디안이란 무엇인가?

2.1 라디안의 정의

라디안(radian)은 각도를 나타내는 단위 중 하나이다. 우리가 일상에서 많이 사용하는 각도 단위는 도(degree)이고, 수학이나 로봇 제어에서는 라디안(rad)을 많이 사용한다.
라디안은 원에서 호의 길이와 반지름의 관계로 정의된다.

라디안 = 호의 길이 / 반지름
 $\theta = s / r$

기호	의미
θ	각도, 라디안
s	호의 길이
r	반지름

즉, 원에서 호의 길이가 반지름과 같을 때의 중심각을 1라디안이라고 한다.

2.2 라디안과 도(degree)의 관계

원 한 바퀴는 360도이다. 라디안으로는 원 한 바퀴가 2π rad 이다.

$360^\circ = 2\pi \text{ rad}$
 $180^\circ = \pi \text{ rad}$
 $90^\circ = \pi/2 \text{ rad}$

따라서 도와 라디안은 서로 변환할 수 있다.

각도(degree)	라디안(rad)
0°	0 rad
90°	$\pi/2$ rad
180°	π rad
270°	$3\pi/2$ rad
360°	2π rad

2.3 도와 라디안 변환 공식

도를 라디안으로 바꿀 때는 다음 공식을 사용한다.

$$\text{라디안} = \text{도} \times \pi / 180$$

라디안을 도로 바꿀 때는 다음 공식을 사용한다.

$$\text{도} = \text{라디안} \times 180 / \pi$$

예를 들어 90도를 라디안으로 바꾸면 다음과 같다.

$$90^\circ \times \pi / 180 = \pi/2 \text{ rad}$$

반대로 π rad 를 도로 바꾸면 다음과 같다.

$$\pi \times 180 / \pi = 180^\circ$$

로봇 제어에서는 회전 각도나 각속도를 계산할 때 라디안을 사용하는 경우가 많다. 예를 들어 로봇이 제자리에서 90도 회전하도록 제어하려면 내부 계산에서는 $\pi/2$ rad 를 사용하는 경우가 많다.

3. 차동 구동 방식의 개념

3.1 차동 구동 방식의 정의

차동 구동(differential drive)은 좌우에 배치된 두 개의 구동 바퀴를 서로 다른 속도로 회전시켜 로봇의 직진, 회전, 제자리 회전을 구현하는 이동 방식이다.

일반적으로 차동 구동 로봇은 다음과 같은 구조를 가진다.

구성 요소	역할
왼쪽 구동 바퀴	왼쪽 방향의 추진력 생성
오른쪽 구동 바퀴	오른쪽 방향의 추진력 생성
보조 바퀴 또는 캐스터	로봇의 균형 유지
모터	각 바퀴를 회전시킴
엔코더	바퀴 회전량과 속도 측정

차동 구동 방식은 구조가 단순하고 제어가 비교적 쉬워서 모바일 로봇, 로봇 청소기, 교육용 로봇, 물류 운반 로봇 등에 많이 사용된다.

3.2 차동 구동의 기본 원리

차동 구동의 핵심은 좌우 바퀴의 속도 차이이다.

왼쪽 바퀴 속도	오른쪽 바퀴 속도	로봇 움직임
같음	같음	직진
왼쪽이 느림	오른쪽이 빠름	왼쪽으로 회전
왼쪽이 빠름	오른쪽이 느림	오른쪽으로 회전
서로 반대 방향	서로 반대 방향	제자리 회전

예를 들어 왼쪽 바퀴와 오른쪽 바퀴가 같은 속도로 앞으로 회전하면 로봇은 직진한다. 하지만 오른쪽 바퀴가 왼쪽 바퀴보다 빠르게 회전하면 로봇의 오른쪽이 더 많이 앞으로 나아가므로 로봇은 왼쪽으로 꺾이게 된다.

즉, 차동 구동은 자동차처럼 조향 바퀴를 꺾는 방식이 아니라, 좌우 바퀴의 속도 차이를 이용해 방향을 바꾸는 방식이다.

4. 각속도의 개념

4.1 각속도란?

각속도(angular velocity)는 물체가 얼마나 빠르게 회전하는지를 나타내는 값이다. 단위는 보통 **rad/s** 를 사용한다.

$$\text{각속도} = \text{회전 각도} / \text{시간}$$
$$\omega = \theta / t$$

기호	의미
ω	각속도
θ	회전 각도
t	시간

예를 들어 바퀴가 1초 동안 2π rad 만큼 회전했다면, 바퀴는 1초에 한 바퀴 회전한 것이다.

$$\omega = 2\pi \text{ rad/s}$$

4.2 바퀴의 각속도와 선속도

바퀴가 회전하면 로봇은 앞으로 이동한다. 이때 바퀴의 회전 속도인 각속도와 로봇이 앞으로 이동하는 속도인 선속도 사이에는 다음 관계가 있다.

$$v = r\omega$$

기호	의미
v	선속도, m/s
r	바퀴 반지름, m
ω	바퀴 각속도, rad/s

즉, 바퀴 반지름이 크거나 바퀴 각속도가 빠르면 로봇의 이동 속도도 빨라진다.

예를 들어 바퀴 반지름이 0.05m 이고, 바퀴 각속도가 10 rad/s 라면 선속도는 다음과 같다.

$$v = 0.05 \times 10 = 0.5 \text{ m/s}$$

5. 회전 반경의 개념

회전 반경(turning radius)은 로봇이 회전할 때 중심이 그리는 원의 반지름을 의미한다. 쉽게 말하면 로봇이 얼마나 크게 도는지를 나타내는 값이다.

회전 반경이 크면 로봇은 완만하게 회전하고, 회전 반경이 작으면 로봇은 급하게 회전한다.

회전 반경	의미
큼	넓게, 완만하게 회전
작음	좁게, 급하게 회전
0	제자리 회전

차동 구동 로봇에서는 좌우 바퀴의 속도 차이가 클수록 회전 반경이 작아지고, 속도 차이가 작을수록 회전 반경이 커진다.

좌우 바퀴 속도 차이가 작다 → 방향이 천천히 바뀐다 → 큰 원을 그리며 돈다 → 회전 반경이 크다
좌우 바퀴 속도 차이가 크다 → 방향이 빠르게 바뀐다 → 작은 원을 그리며 돈다 → 회전 반경이 작다

6. 각 바퀴의 각속도가 이동 경로에 미치는 영향

차동 구동 로봇에서 각 바퀴의 각속도는 이동 경로를 결정한다. 바퀴 반지름이 같다고 가정하면, 왼쪽 바퀴와 오른쪽 바퀴의 각속도 차이가 곧 좌우 바퀴의 선속도 차이로 이어진다.

왼쪽 바퀴의 선속도를 v_L , 오른쪽 바퀴의 선속도를 v_R 이라고 하면 다음과 같이 해석할 수 있다.

조건	이동 경로
$v_L = v_R$	직선 이동
$v_L < v_R$	왼쪽으로 회전
$v_L > v_R$	오른쪽으로 회전
$v_L = -v_R$	제자리 회전

예를 들어 오른쪽 바퀴가 왼쪽 바퀴보다 빠르게 움직이면 오른쪽 바퀴가 더 긴 거리를 이동한다. 따라서 로봇은 왼쪽 방향으로 휘어진 경로를 따라 움직인다.

이때 로봇의 중심 속도는 좌우 바퀴 속도의 평균으로 볼 수 있다.

$$\text{로봇 중심 선속도} = (v_R + v_L) / 2$$

또한 로봇의 회전 각속도는 좌우 바퀴 속도의 차이와 바퀴 간 거리로 계산할 수 있다.

$$\text{로봇 회전 각속도} = (v_R - v_L) / L$$

여기서 L 은 좌우 바퀴 사이의 거리, 즉 휠 베이스 또는 트랙 폭을 의미한다.

7. 두 바퀴의 속도 차이에 따른 회전 반경 및 회전 방향의 변화

차동 구동 로봇의 회전 방향은 좌우 바퀴 속도 차이에 의해 결정된다.

7.1 오른쪽 바퀴가 더 빠른 경우

$$v_R > v_L$$

오른쪽 바퀴가 왼쪽 바퀴보다 빠르면 오른쪽 바퀴가 더 많이 앞으로 이동한다. 이 경우 로봇은 왼쪽으로 회전한다.

오른쪽 바퀴 빠름 → 왼쪽으로 회전

7.2 왼쪽 바퀴가 더 빠른 경우

$$v_L > v_R$$

왼쪽 바퀴가 오른쪽 바퀴보다 빠르면 왼쪽 바퀴가 더 많이 앞으로 이동한다. 이 경우 로봇은 오른쪽으로 회전한다.

왼쪽 바퀴 빠름 → 오른쪽으로 회전

7.3 두 바퀴 속도가 같은 경우

$$v_L = v_R$$

좌우 바퀴가 같은 속도로 움직이면 로봇은 회전하지 않고 직진한다. 이때 회전 반경은 무한대에 가깝다고 볼 수 있다.

Q

좌우 바퀴가 같은 속도로 움직이면 로봇은 회전하지 않고 직진한다. 이때 회전 반경은 무한대에 가깝다고 볼 수 있다. 회전반경이 무한대라는 의미가 이해가 안가는데?

A

1. 회전 반경이 점점 커지면 어떻게 될까?

원이 작으면 확 꺾여.

작은 원: 급회전

원이 커지면 덜 꺾여.

큰 원: 완만한 회전

원이 엄청나게 커지면?

엄청 큰 원: 거의 직선처럼 보임

즉, 반지름이 아주 큰 원의 일부분은 거의 직선처럼 보인다는 거야.

1. 차동 구동에서 보면

회전 반경 공식은 보통 이렇게 써.

$$R = v / \omega$$

여기서:

R = 회전 반경 v = 로봇 중심의 직진 속도 ω = 로봇이 도는 각속도

좌우 바퀴 속도가 같으면 로봇은 회전하지 않아.

속도 차이 없음 → 직진

7.4 두 바퀴 속도 차이가 큰 경우

좌우 바퀴의 속도 차이가 커질수록 로봇은 더 급하게 회전한다. 즉, 회전 반경이 작아진다.

속도 차이 작음 → 큰 원을 그리며 회전

속도 차이 큼 → 작은 원을 그리며 회전

8. 제자리 회전 방법

제자리 회전은 로봇의 중심 위치는 거의 변하지 않고, 로봇이 그 자리에서 방향만 바꾸는 회전이다. 차동 구동 로봇에서는 좌우 바퀴를 서로 반대 방향으로 같은 속도로 회전시키면 제자리 회전이 가능하다.

예를 들어 다음과 같이 제어할 수 있다.

왼쪽 바퀴	오른쪽 바퀴	결과
앞으로 회전	뒤로 회전	한쪽 방향으로 제자리 회전
뒤로 회전	앞으로 회전	반대 방향으로 제자리 회전

수식으로 표현하면 다음과 같다.

$$v_L = -v_R$$

이 경우 로봇 중심의 선속도는 다음과 같다.

$$v = (v_R + v_L) / 2 = 0$$

즉, 중심은 앞으로 이동하지 않는다. 하지만 좌우 바퀴가 서로 반대 방향으로 움직이기 때문에 로봇은 회전하게 된다.

9. 바퀴 간 거리, 즉 휠 베이스가 미치는 영향

차동 구동 로봇에서 바퀴 간 거리 L 은 회전 성능에 큰 영향을 준다. 여기서 L 은 왼쪽 바퀴와 오른쪽 바퀴 사이의 거리이다.

휠 베이스	특징
짧음	같은 속도 차이에서도 더 빠르게 회전한다. 회전 반경이 작아진다.
김	같은 속도 차이에서도 천천히 회전한다. 회전 반경이 커진다.

로봇의 회전 각속도 공식은 다음과 같다.

$$\omega = (vR - vL) / L$$

이 공식에서 L 이 커지면 같은 속도 차이에서도 ω 가 작아진다. 즉, 로봇이 천천히 회전한다. 반대로 L 이 작으면 ω 가 커져서 로봇이 더 빠르게 회전한다.

예를 들어 같은 속도 차이를 가진 두 로봇이 있다고 가정한다.

참고(두종류의 각속도 존재)

바퀴 각속도 = 바퀴가 빙글빙글 도는 속도 로봇 각속도 = 로봇 전체가 왼쪽/오른쪽으로 도는 속도

로봇 회전 각속도 = 바퀴 속도 차이 / 바퀴 사이 거리 $\omega = (vR - vL) / L$

식	의미	대상
$\omega = \theta/t$	회전각도를 시간으로 나눈 것	바퀴, 모터축 같은 회전체
$\omega = (vR - vL)/L$	좌우 속도 차이로 생기는 회전 속도	로봇 몸체 전체

$$\text{속도 차이} = vR - vL = 0.4 \text{ m/s}$$

휠 베이스가 0.2m 라면:

$$\omega = 0.4 / 0.2 = 2 \text{ rad/s}$$

휠 베이스가 0.4m 라면:

$$\omega = 0.4 / 0.4 = 1 \text{ rad/s}$$

따라서 바퀴 간 거리가 짧은 로봇이 같은 속도 차이에서도 더 빠르게 회전한다.

10. 회전 반경을 계산하는 방법

차동 구동 로봇의 회전 반경은 로봇 중심 선속도와 회전 각속도를 이용해 계산할 수 있다.

로봇 중심 선속도는 다음과 같다.

$$v = (vR + vL) / 2$$

로봇 회전 각속도는 다음과 같다.

$$\omega = (vR - vL) / L$$

회전 반경은 다음과 같이 계산한다.

$$R = v / \omega$$

위 식을 정리하면 다음과 같다.

$$R = L / 2 \times (vR + vL) / (vR - vL)$$

기호	의미
R	로봇 중심의 회전 반경
L	좌우 바퀴 사이 거리
vR	오른쪽 바퀴 선속도
vL	왼쪽 바퀴 선속도

10.1 회전 반경 계산 예시

로봇의 휠 베이스가 0.4m 이고, 왼쪽 바퀴 속도와 오른쪽 바퀴 속도가 다음과 같다고 가정한다. $v = (vR + vL) / 2$

$$\omega = (vR - vL) / L$$

$$R = v / \omega$$

$$\begin{aligned} L &= 0.4m \\ vL &= 0.2m/s \\ vR &= 0.6m/s \end{aligned}$$

먼저 로봇 중심 선속도를 계산한다.

$$\begin{aligned} v &= (vR + vL) / 2 \\ v &= (0.6 + 0.2) / 2 \\ v &= 0.4m/s \end{aligned}$$

다음으로 회전 각속도를 계산한다.

$$\begin{aligned} \omega &= (vR - vL) / L \\ \omega &= (0.6 - 0.2) / 0.4 \\ \omega &= 1 \text{ rad/s} \end{aligned}$$

마지막으로 회전 반경을 계산한다.

$$\begin{aligned} R &= v / \omega \\ R &= 0.4 / 1 \\ R &= 0.4m \end{aligned}$$

따라서 이 로봇은 중심 기준으로 반지름 0.4m 인 원을 그리며 회전한다. 이때 오른쪽 바퀴가 더 빠르므로 로봇은 왼쪽으로 회전한다.

10.2 특수한 경우

차동 구동에서는 몇 가지 특수한 경우가 있다.

조건	결과	회전 반경
$v_L = v_R$	직진	무한대
$v_L = -v_R$	제자리 회전	0
$v_L = 0, v_R > 0$	왼쪽 바퀴를 중심으로 회전	약 $L/2$
$v_R = 0, v_L > 0$	오른쪽 바퀴를 중심으로 회전	약 $L/2$

직진의 경우 좌우 바퀴 속도 차이가 없으므로 회전 각속도는 0이다. 따라서 회전 반경은 무한대에 가까운 값으로 볼 수 있다.

제자리 회전의 경우 중심 선속도는 0이지만 회전 각속도는 존재한다. 따라서 로봇의 중심은 거의 이동하지 않고 방향만 바뀐다.

##Q. 선속도는 앞으로가는 거리를 선속도라고 하지않나?

##A. Chatgpt 답변 맞아, 네 감각이 거의 맞아. 다만 "거리"와 "속도"를 구분해야 해.

정확히는:

선속도 = 앞으로 가는 거리 / 시간

즉, 앞으로 간 거리 자체가 아니라, 앞으로 얼마나 빠르게 가는지가 선속도야.

11. 로봇 운반차에서 차동 구동을 사용할 때 고려사항

차동 구동 로봇을 설계하거나 제어할 때는 다음 사항을 고려해야 한다.

고려사항	설명
좌우 바퀴 속도 제어	좌우 모터 속도가 정확히 제어되어야 원하는 방향으로 이동할 수 있다.
엔코더 사용	각 바퀴의 실제 속도와 이동 거리를 측정하여 보정할 수 있다.
바퀴 미끄러짐	미끄러운 바닥에서는 계산한 경로와 실제 경로가 달라질 수 있다.
휠 베이스 측정	바퀴 간 거리 L 이 회전 계산에 직접 영향을 준다.
바퀴 지름 차이	좌우 바퀴 지름이 다르면 직진 명령에서도 한쪽으로 휘어질 수 있다.
하중 분포	무게가 한쪽으로 치우치면 마찰력 차이로 주행 오차가 발생할 수 있다.
바닥 상태	카펫, 경사면, 요철 등은 회전 반경과 직진성에 영향을 준다.
제어 주기	너무 느린 제어 주기는 속도 보정과 방향 제어를 어렵게 만든다.

12. 결론

차동 구동 방식은 좌우 두 개의 바퀴 속도 차이를 이용하여 로봇의 이동 방향을 제어하는 방식이다. 두 바퀴가 같은 속도로 움직이면 로봇은 직진하고, 한쪽 바퀴가 더 빠르면 반대쪽 방향으로 회전한다. 또한 좌우 바퀴를 서로 반대 방향으로 같은 속도로 회전시키면 제자리 회전이 가능하다.

라디안은 회전 각도를 표현하는 단위이며, 로봇의 회전 각속도 계산에서 자주 사용된다. 바퀴 각속도와 바퀴 반지름을 이용하면 바퀴의 선속도를 구할 수 있고, 좌우 바퀴 선속도를 이용하면 로봇 중심 속도, 회전 각속도, 회전 반경을 계산할 수 있다.

선속도 = 이동 거리 / 시간 $v = s / t$ 로봇이 2초 동안 1m 이동했다면 선속도 = $1\text{m} / 2\text{초} = 0.5\text{m/s}$

각속도 = 회전 각도 / 시간 $\omega = \theta / t$ 바퀴가 1초 동안 180도 회전했다면 $180\text{도} = \pi \text{ rad}$ 각속도 = $\pi \text{ rad} / 1\text{초} = \pi \text{ rad/s}$

차동 구동에서 회전 반경은 좌우 바퀴 속도 차이와 휠 베이스에 의해 결정된다. 속도 차이가 클수록 회전 반경은 작아지고, 휠 베이스가 클수록 같은 속도 차이에서도 회전은 느려진다.

따라서 차동 구동 로봇을 정확하게 제어하기 위해서는 라디안, 각속도, 선속도, 회전 반경, 휠 베이스의 개념을 함께 이해해야 하며, 실제 주행에서는 엔코더, 바퀴 미끄러짐, 바닥 상태, 좌우 모터 차이 등을 고려하여 보정해야 한다.

휠베이스 추가설명

자동차에서 휠베이스

자동차에서는 보통:

휠베이스 = 앞바퀴 축과 뒷바퀴 축 사이 거리

즉 옆에서 봤을 때 이 거리야.

앞바퀴 ○ ----- 휠베이스 ----- ○ 뒷바퀴

자동차는 앞뒤 바퀴가 있으니까 휠베이스가 앞축과 뒤축 사이 거리를 뜻해.

차동 구동 로봇에서 말하는 L

그런데 우리가 지금 배우는 두 바퀴 차동 구동 로봇에서는 보통 좌우 바퀴가 있지?

왼쪽 바퀴 ○ ----- L ----- ○ 오른쪽 바퀴

여기서 공식에 들어가는 L은:

좌우 구동 바퀴 중심 사이 거리

야.

정확한 용어로는 자동차식 휠베이스보다는 트랙 폭(track width) 또는 wheel separation이라고 하는 게 더 정확해.

13. 참고자료

소제목	참고 링크
2. 라디안의 정의와 변환	https://en.wikipedia.org/wiki/Radian
3. 차동 구동 방식의 개념	https://en.wikipedia.org/wiki/Differential_wheeled_robot
4. 각속도의 개념	https://en.wikipedia.org/wiki/Angular_velocity
5. 회전 반경의 개념	https://en.wikipedia.org/wiki/Turning_radius
6. 바퀴 각속도와 이동 경로	https://automaticaddison.com/calculating-wheel-velocities-for-a-differential-drive-robot/
7. 두 바퀴 속도 차이와 회전 방향	https://www.cs.columbia.edu/~allen/F17/NOTES/icckinematics.pdf
8. 제자리 회전 방법	https://www.roboticsbook.org/S52_diffdrive_actions.html
9. 휠 베이스가 회전에 미치는 영향	https://www.roboticsbook.org/S52_diffdrive_actions.html
10. 회전 반경 계산 방법	https://www.cs.columbia.edu/~allen/F17/NOTES/icckinematics.pdf
11. 차동 구동 로봇 제어 고려사항	https://control.ros.org/master/doc/ros2_controllers/diff_drive_controller/doc/userdoc.html

ROS2 노드(Node)와 rqt_graph 실습

1. 수행목표

ROS2에서 노드(node)의 개념을 학습하고, ROS2 데모 프로그램인 `talker`, `listener` 노드와 `turtlesim_node`, `turtle_teleop_key` 노드를 실행하여 노드 간의 관계를 확인한다. 또한 `rqt_graph`, `ros2 node list`, `ros2 node info` 명령을 사용하여 ROS2 노드의 연결 구조와 정보를 확인한다.

2. ROS2에서 노드란 무엇인가?

ROS2에서 노드(node)는 하나의 기능을 수행하는 실행 단위이다. 로봇 시스템은 여러 기능으로 나누어져 있는데, 각각의 기능을 하나의 노드로 만들어 실행할 수 있다.

예를 들어 이동 로봇을 만든다고 하면 다음과 같이 여러 노드로 나눌 수 있다.

노드 예시	역할
카메라 노드	카메라 영상을 읽는다.
라이다 노드	라이다 거리 데이터를 읽는다.
모터 제어 노드	바퀴 모터를 제어한다.
위치 추정 노드	센서 데이터를 이용해 로봇 위치를 계산한다.
경로 계획 노드	목적지까지 이동 경로를 계산한다.

즉, ROS2에서 노드는 로봇을 구성하는 각각의 프로그램 단위라고 볼 수 있다.

3. 노드가 필요한 이유

로봇은 센서, 모터, 카메라, 라이다, 알고리즘 등 여러 기능이 동시에 동작해야 한다. 하나의 큰 프로그램 안에 모든 기능을 넣으면 관리가 어렵고, 오류가 발생했을 때 원인을 찾기 어렵다.

ROS2에서는 기능을 여러 노드로 나누어 실행할 수 있다. 이렇게 하면 다음과 같은 장점이 있다.

장점	설명
기능 분리	센서, 제어, 판단 기능을 각각 따로 만들 수 있다.
유지보수 쉬움	문제가 생긴 노드만 수정할 수 있다.
재사용 가능	이미 만든 노드를 다른 로봇에서도 사용할 수 있다.
동시 실행 가능	여러 노드가 동시에 동작하면서 데이터를 주고받을 수 있다.
분산 실행 가능	여러 컴퓨터에서 노드를 나누어 실행할 수 있다.

따라서 ROS2에서 노드는 로봇 시스템을 구성하는 기본 단위이다.

4. ROS2 노드 간 통신

ROS2 노드는 혼자 동작하기도 하지만, 보통 다른 노드와 데이터를 주고받으면서 동작한다. 대표적인 통신 방식은 토픽(topic), 서비스(service), 액션(action)이 있다.

통신 방식	설명	예시
토픽	한 노드가 데이터를 보내고 다른 노드가 구독하는 방식	카메라 영상, 센서값, 속도 명령
서비스	요청과 응답 방식의 통신	특정 기능 실행 요청
액션	시간이 오래 걸리는 작업을 목표, 피드백, 결과로 처리	목적지 이동, 로봇팔 동작

이번 실습에서 사용하는 `talker`와 `listener`는 토픽 통신을 사용한다.

```
talker 노드 → /chatter 토픽 → listener 노드
```

`turtlesim_node`와 `turtle_teleop_key`도 토픽 통신을 사용한다.

```
turtle_teleop_key 노드 → /turtle1/cmd_vel 토픽 → turtlesim_node 노드
```

5. rqt_graph란?

`rqt_graph`는 ROS2에서 실행 중인 노드와 토픽의 관계를 그래프 형태로 보여주는 도구이다. 즉, 어떤 노드가 어떤 토픽을 발행하고, 어떤 노드가 그 토픽을 구독하는지 시각적으로 확인할 수 있다.

실행 명령은 다음과 같다.

```
rqt_graph
```

`rqt_graph`를 실행하면 노드와 토픽 관계가 그래프 형태로 나타난다. 노드 간의 관계를 확인하려면 보기 옵션에서 `Nodes only` 모드를 사용할 수 있다. 만약 최근 실행 상태가 바로 보이지 않으면 리로드 버튼을 눌러 화면을 새로 고친다.

6. talker/listener 노드 실행

6.1 실행 전 환경 설정

ROS2 명령을 사용하려면 터미널에서 ROS2 환경 설정이 적용되어 있어야 한다.

```
source /opt/ros/humble/setup.bash
```


매번 터미널을 열 때 자동으로 적용하고 싶다면 다음 명령을 사용할 수 있다.

```
echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc
source ~/.bashrc
```

환경 설정이 정상적으로 적용되었는지 확인하려면 다음 명령을 사용한다.

```
printenv ROS_DISTRO
```

정상적으로 설정되어 있다면 다음과 같이 출력된다.

```
humble
```

6.2 talker 노드 실행

첫 번째 터미널에서 다음 명령을 실행한다.

```
ros2 run demo_nodes_cpp talker
```

`talker` 노드는 `/chatter` 토픽으로 `Hello World` 메시지를 계속 발행한다.

예상 출력은 다음과 같다.

```
[INFO] [talker]: Publishing: 'Hello World: 1'
[INFO] [talker]: Publishing: 'Hello World: 2'
[INFO] [talker]: Publishing: 'Hello World: 3'
```

6.3 listener 노드 실행

두 번째 터미널에서 다음 명령을 실행한다.

```
ros2 run demo_nodes_cpp listener
```

`listener` 노드는 `/chatter` 토픽을 구독하여 `talker` 가 보낸 메시지를 출력한다.

예상 출력은 다음과 같다.

```
[INFO] [listener]: I heard: [Hello World: 1]
[INFO] [listener]: I heard: [Hello World: 2]
[INFO] [listener]: I heard: [Hello World: 3]
```

6.4 talker/listener 노드 관계

`talker` 와 `listener` 의 관계는 다음과 같다.

구성 요소	역할
talker 노드	메시지를 발행하는 노드
/chatter 토픽	두 노드 사이에서 메시지가 이동하는 통신 통로
listener 노드	메시지를 구독하는 노드

관계는 다음과 같이 표현할 수 있다.

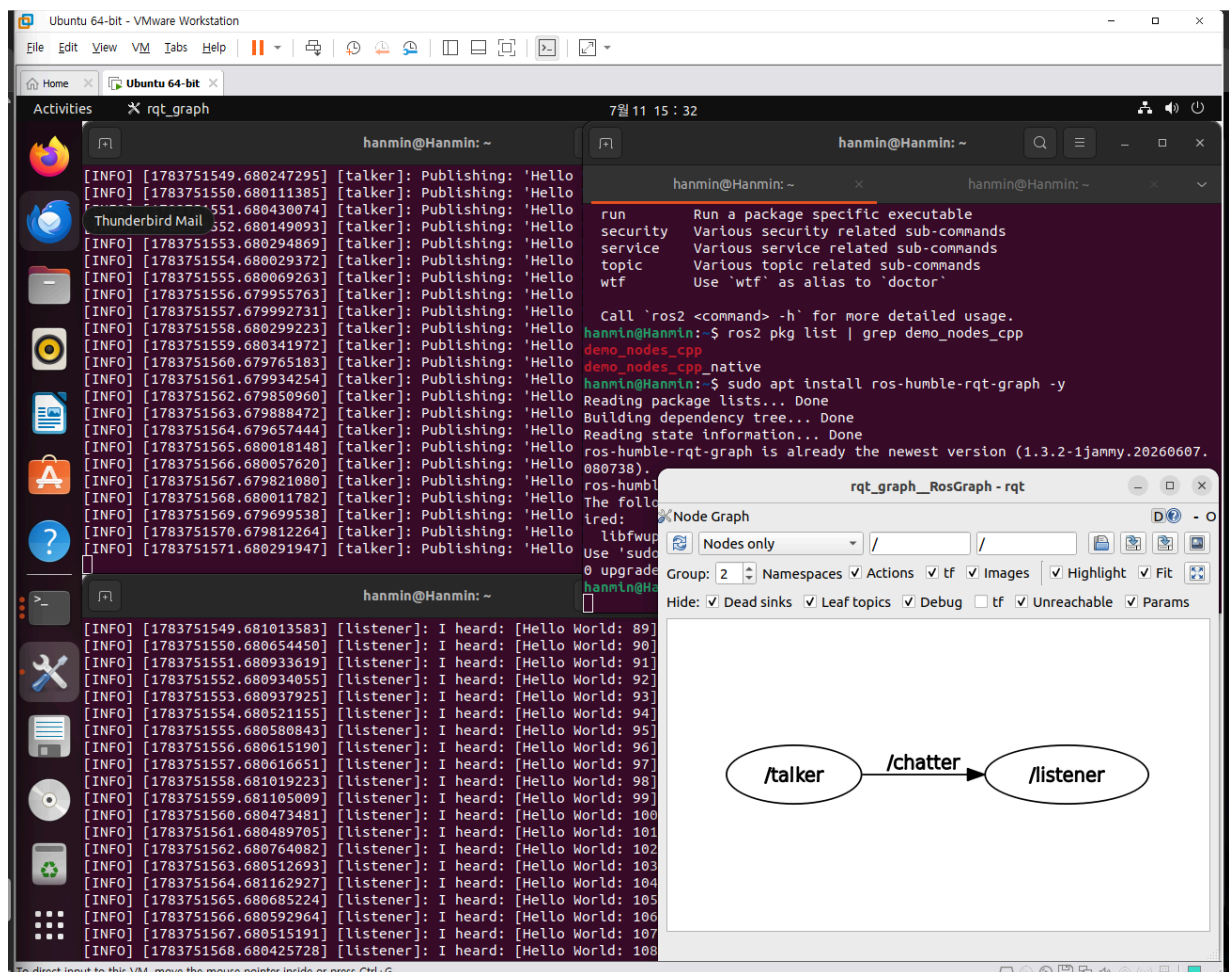
```
/talker → /chatter → /listener
```

7. talker/listener 실행 상태에서 rqt_graph 확인

talker 와 listener 를 실행한 상태에서 새 터미널을 열고 다음 명령을 실행한다.

```
rqt_graph
```

rqt_graph 화면에서 Nodes only 모드를 선택하면 /talker 노드와 /listener 노드의 관계를 확인할 수 있다.



위 캡처에서 /talker 노드가 /chatter 토픽을 발행하고, /listener 노드가 해당 토픽을 구독하는 구조를 확인할 수 있다.

8. ros2 node list 명령

`ros2 node list` 명령은 현재 실행 중인 ROS2 노드 목록을 확인하는 명령이다.

실행 명령은 다음과 같다.

```
ros2 node list
```

`talker`와 `listener`가 실행 중이라면 다음과 비슷한 결과가 나온다.

```
/listener  
/talker
```

이 결과는 `rqt_graph`에서 보이는 노드 목록과 일치해야 한다. 즉, `rqt_graph`에서 `/talker`, `/listener`가 보인다면 `ros2 node list`에서도 같은 노드가 출력되어야 한다.

9. ros2 node info 명령

`ros2 node info` 명령은 특정 노드가 어떤 토픽을 발행하고 구독하는지, 어떤 서비스와 액션을 사용하는지 확인하는 명령이다.

기본 형식은 다음과 같다.

```
ros2 node info /노드이름
```

9.1 talker 노드 정보 확인

```
ros2 node info /talker
```

예상되는 주요 내용은 다음과 같다.

항목	설명
Publications	노드가 발행하는 토픽 목록
Subscriptions	노드가 구독하는 토픽 목록
Services	노드가 제공하거나 사용하는 서비스 목록

`talker`는 `/chatter` 토픽을 발행한다.

```
/talker → /chatter
```

9.2 listener 노드 정보 확인

```
ros2 node info /listener
```

listener 는 /chatter 토픽을 구독한다.

```
/chatter → /listener
```

따라서 talker 와 listener 는 /chatter 토픽을 통해 연결된다.

10. turtlesim 노드 실행

turtlesim 은 ROS2와 함께 제공되는 간단한 데모 로봇 패키지이다. 화면에 거북이 로봇이 나타나고, 키보드로 거북이를 움직일 수 있다.

기존에 실행 중인 talker 와 listener 를 종료한 후 turtlesim 실습을 진행한다.

종료는 각 터미널에서 다음 키를 누르면 된다.

```
Ctrl + C
```

10.1 turtlesim_node 실행

첫 번째 터미널에서 다음 명령을 실행한다.

```
ros2 run turtlesim turtlesim_node
```

그러면 turtlesim 창이 열리고 거북이 로봇이 화면에 나타난다.

10.2 turtle_teleop_key 실행

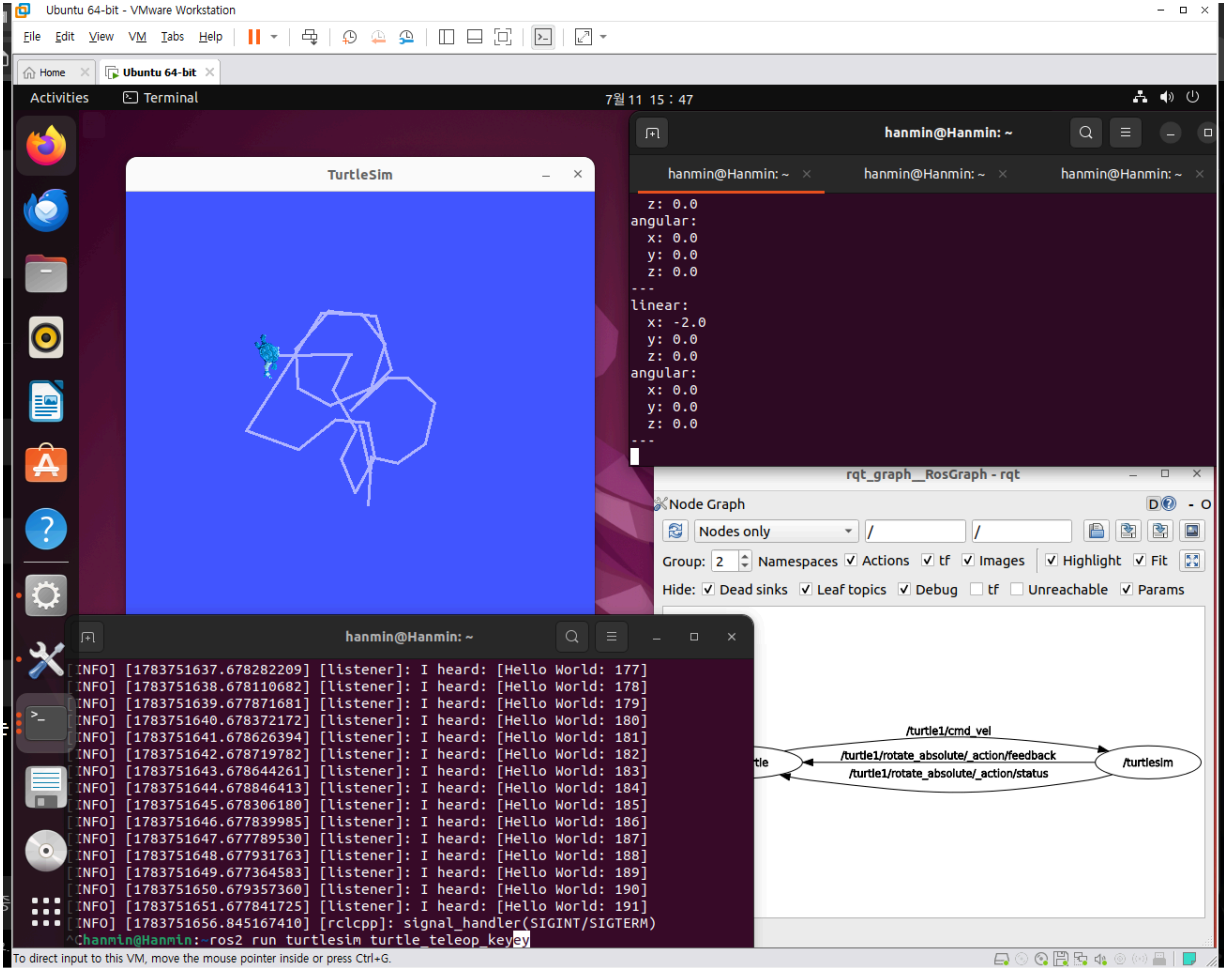
두 번째 터미널에서 다음 명령을 실행한다.

```
ros2 run turtlesim turtle_teleop_key
```

이 노드는 키보드 입력을 받아 거북이에게 속도 명령을 보낸다.

실행 후 방향키를 누르면 거북이가 움직인다.

키	동작
↑	앞으로 이동
↓	뒤로 이동
←	왼쪽 회전
→	오른쪽 회전



11. turtlesim 노드 간 관계

turtlesim_node와 turtle_teleop_key의 관계는 다음과 같다.

구성 요소	역할
turtlesim_node	거북이 로봇을 화면에 표시하고 움직임을 처리하는 노드
turtle_teleop_key	키보드 입력을 받아 속도 명령을 보내는 노드
/turtle1/cmd_vel	거북이에게 전달되는 속도 명령 토픽

관계는 다음과 같이 표현할 수 있다.

구분	이름	역할	방향
노드	/teleop_turtle	키보드 입력을 받아 거북이 제어 명령을 생성하는 노드	명령 송신
노드	/turtlesim	거북이 시뮬레이션을 실행하고 명령을 받아 움직이는 노드	명령 수신
토픽	/turtle1/cmd_vel	거북이의 선속도와 각속도 명령을 전달하는 토픽	/teleop_turtle → /turtlesim
액션 피드백	/turtle1/rotate_absolute/_action/feedback	절대 각도 회전 동작의 진행 상황을 전달	/turtlesim → /teleop_turtle
액션 상태	/turtle1/rotate_absolute/_action/status	절대 각도 회전 동작의 현재 상태를 전달	/turtlesim → /teleop_turtle

```
/turtle_teleop_key → /turtle1/cmd_vel → /turtlesim
```

즉, 사용자가 키보드 방향키를 누르면 `turtle_teleop_key` 노드가 속도 명령을 만들고, 이 명령이 `/turtle1/cmd_vel` 토픽을 통해 `turtlesim_node`에 전달된다. `turtlesim_node`는 이 명령을 받아 화면 속 거북이를 움직인다.

12. turtlesim 실행 상태에서 rqt_graph 확인

`turtlesim_node`와 `turtle_teleop_key`를 실행한 상태에서 새 터미널을 열고 다음 명령을 실행한다.

```
rqt_graph
```

`Nodes only` 모드를 선택하거나 옵션을 조정하여 두 노드 사이의 관계가 잘 보이도록 설정한다. 화면이 갱신되지 않으면 리로드 버튼을 누른다.

이때 그래프에는 `turtle_teleop_key`가 `/turtle1/cmd_vel` 토픽을 발행하고, `turtlesim_node`가 이를 구독하는 흐름이 나타난다.

13. ros2 node list로 turtlesim 노드 확인

`turtlesim_node`와 `turtle_teleop_key`가 실행 중인 상태에서 다음 명령을 실행한다.

```
ros2 node list
```

예상 결과는 다음과 같다.

```
/teleop_turtle
/turtlesim
```

실제 출력 이름은 ROS2 버전이나 실행 상태에 따라 약간 다를 수 있다. 중요한 것은 `rqt_graph`에 보이는 노드와 `ros2 node list` 명령 결과가 일치하는지 확인하는 것이다.

14. ros2 node info로 turtlesim 노드 정보 확인

14.1 turtlesim 노드 정보

```
ros2 node info /turtlesim
```

`turtlesim` 노드는 `/turtle1/cmd_vel` 토픽을 구독하여 거북이를 움직인다.

14.2 teleop 노드 정보

```
ros2 node info /teleop_turtle
```

`teleop_turtle` 노드는 키보드 입력을 바탕으로 `/turtle1/cmd_vel` 토픽을 발행한다.
정리하면 다음과 같다.

```
/teleop_turtle → /turtle1/cmd_vel → /turtlesim
```

15. 실습 결과 정리

이번 실습에서는 ROS2의 노드 개념과 노드 간 통신 구조를 확인하였다.

첫 번째로 `demo_nodes_cpp` 패키지의 `talker` 노드와 `listener` 노드를 실행하였다. `talker`는 `/chatter` 토픽으로 메시지를 발행하고, `listener`는 `/chatter` 토픽을 구독하여 메시지를 수신하였다. `rqt_graph`와 `ros2 node list`를 통해 두 노드가 실행 중임을 확인할 수 있었다.

두 번째로 `turtlesim` 패키지의 `turtlesim_node`와 `turtle_teleop_key` 노드를 실행하였다. `turtle_teleop_key`는 키보드 입력을 받아 `/turtle1/cmd_vel` 토픽으로 속도 명령을 발행하고, `turtlesim_node`는 이 명령을 받아 거북이 로봇을 움직였다.

이를 통해 ROS2에서 노드는 각각의 기능을 담당하는 실행 단위이며, 노드들은 토픽을 통해 데이터를 주고받으면서 하나의 로봇 시스템을 구성한다는 것을 확인할 수 있었다.

16. 참고자료

소제목	참고 링크
ROS2 Humble 공식 문서	https://docs.ros.org/en/humble/
ROS2 노드 개념	https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Nodes/Understanding-ROS2-Nodes.html
ROS2 토픽 개념	https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Topics/Understanding-ROS2-Topics.html
rqt_graph 사용	https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Introspection-With-Rqt-Graph/Introspection-With-Rqt-Graph.html
ros2 node 명령	https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Nodes/Understanding-ROS2-Nodes.html
turtlesim 튜토리얼	https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Introducing-Turtlesim/Introducing-Turtlesim.html